

Título del documento a varias líneas

Subtítulo del documento a varias líneas

MEDAC
Instituto Oficial de Formación Profesional

Índice

1. **Parte 1: Análisis teórico de conceptos**
2. **Clasificación de lenguajes de programación:**
3. **Parte 2: Actividad práctica y de análisis**
4. **Identificación de paradigmas de programación a partir de ejemplos:**
5. **Fragmento 1 (Descripción):**
6. **Fragmento 2 (Descripción):**
7. **Fragmento 3 (Descripción):**
8. **Actividad en grupo:**

Parte 1: Análisis teórico de conceptos.

Explicación de los conceptos básicos:

- **Código fuente**: Es la parte que está escrita por el programador dentro de un lenguaje de programación, como python, java, c + +, que al ser ejecutada realiza las instrucciones del código. Por ejemplo:

Numero = 5

Resultado = numero *2

print(resultado)

Ese ejemplo formaría parte del código fuente en python escrito por el programador por ejemplo, donde asignamos a número el 5 y a resultado número por 2, para que imprima la solución que sería 10.

Cabe resaltar que el código fuente suele guardarse en archivos con extensiones para cada lenguaje, para python sería = .py, Java = Java, JavaScript= .js .

- **Código Objeto:** Se encarga de traducir el código fuente a una manera que el procesador entiende para que se pueda ejecutar, y esto es el proceso de compilación. El código fuente está hecho para que lo entienda una persona por así decirlo y el código objeto está hecho para que lo entienda la máquina.

Por ejemplo: Código fuente pasa al compilador y de el compilador a código objeto y de ahí también puede enlazarse con otras partes del programa.

- **Código ejecutable:** Es la última parte que se enlaza con las anteriores, por ejemplo la preparación de un plato, los ingredientes serían código fuente, la preparación del mismo sería código fuente y el plato terminado sería el código ejecutable porque está listo para ejecutarse.

Ejemplo: Código fuente se compila→ Código objeto , se enlaza → Código ejecutable llega al procesador.

Clasificación y fases de lenguajes de programación:

- Todos los programas atraviesan diferentes fases desde que es escrito hasta que es ejecutado y todo sigue un esquema:

1. **Escritura del código fuente:** usaré el ejemplo anterior de python:

Numero = 5

Resultado = numero *2

print(resultado)

El programador desarrolla el programa en un determinado lenguaje.

2. **Análisis léxico:** El código fuente se divide en elementos básicos llamados tokens. Por ejemplo: numero, =,5, se divide en palabras reservadas, identificadores, símbolos, números, cadenas de texto, etc. En caso de que haya un carácter que no pertenezca al lenguaje puede dar un error léxico al compilarse o ejecutarse.

numero → identificador
= → operador de asignación
5 → literal numérico

3. **Análisis sintáctico:** Se encarga de asegurarse que los elementos o el código esté bien organizado en base a las reglas de su lenguaje, por ejemplo en python:

If edad >=20:

Aquí estaría correcto pero por algo tan simple como 2 puntos puede dar un error de sintaxis, por ejemplo algo incorrecto sería:

If edad >= 20

print(“mayor”)

Al ejecutarse podría dar un error y no tiene porque estar completamente mal.

4. **Análisis semántico**

El **análisis semántico** comprueba que las instrucciones que le hemos dado tienen sentido dentro de las reglas del lenguaje.Por ejemplo:

numero = 10
resultado = numero + 5

es una operación coherente porque edad es un numero y se le puede sumar otro numero.

Sin embargo, una operación incompatible entre determinados tipos de datos puede producir un error semántico también

El análisis semántico también puede comprobar aspectos como:

- tipos de datos
- declaración de variables
- compatibilidad entre operaciones
- utilización correcta de identificadores

5. Compilación o interpretación

Dependiendo del lenguaje utilizado, el código puede ser:

- **Compilado**, transformándose previamente a código objeto o código máquina.
- **Interpretado**, ejecutándose mediante un intérprete.
- Procesado mediante una combinación de ambos métodos.

Por ejemplo, C y C++ utilizan normalmente compilación.

Python utiliza principalmente un intérprete y una máquina virtual, aunque internamente también genera código intermedio (bytecode), que es una especie de borrador rápido o que prepara el sistema para que ese ayudante lea mejor.

6. Enlazado

En programas compilados, el **enlazador** combina diferentes archivos objeto y bibliotecas necesarias para crear el programa final.

Por ejemplo:

programa.o
biblioteca.o
funciones.o
↓
enlazador
↓
programa.exe

7. Carga en memoria

Cuando ejecutamos el programa, el sistema operativo carga el programa ejecutable en la memoria RAM.

El sistema prepara los recursos necesarios para que el programa pueda funcionar.

8. Ejecución

Finalmente, el procesador ejecuta las instrucciones del programa.

El procesador realiza las operaciones necesarias siguiendo las instrucciones que contiene el programa.

De forma simplificada:

Código fuente → Análisis (Léxico/Sintáctico/Semántico) → Compilación (Código objeto) → Enlazado (Ejecutable) → Carga y Ejecución (CPU) → ejecución

Si prefieres mantener todos los pasos individuales en una sola línea:

Código fuente → Léxico → Sintáctico → Semántico → Compilación → Código objeto → Enlazado → Ejecutable → Carga en memoria → Procesador → ejecución

Parte 2: Actividad práctica y de análisis

2.1 Identificación de paradigmas

Fragmento 1

Descripción

Un programa recorre una lista de números sumándolos uno por uno hasta obtener el total.

Clasificación

Imperativo.

Justificación

Es un ejemplo de programación imperativa porque se describe el procedimiento que debe realizar el programa.

El programa:

Recorre la lista.

Obtiene un número.

Lo suma al total.

Continúa con el siguiente número.

Repite el proceso hasta terminar.

Por tanto, se está especificando **cómo realizar la operación paso a paso**.

Fragmento 2

Descripción

Una consulta a una base de datos busca empleados mayores de 30 años y devuelve solamente sus nombres.

Clasificación

Declarativo.

Justificación

Es declarativo porque se especifica el resultado que se quiere obtener:

Los nombres de los empleados cuya edad sea superior a 30 años.

No es necesario indicar cómo debe recorrer la base de datos internamente.

Un ejemplo en SQL sería:

```
SELECT nombre
FROM empleados
WHERE edad > 30;
```

La consulta indica qué información queremos obtener y el sistema de gestión de bases de datos se encarga de decidir cómo realizar la búsqueda.

Fragmento 3

Descripción

Un programa calcula el factorial de un número n definiendo que el factorial de 0 es 1 y que, para números mayores, se multiplica el número por el factorial del número anterior.

Clasificación

Declarativo.

Justificación

En la descripción se define principalmente la relación que debe cumplirse:

0! = 1

n! = n × (n-1)!

La solución se expresa mediante una definición recursiva del problema, en lugar de describir un procedimiento detallado mediante instrucciones paso a paso.

Por ello, dentro de la clasificación planteada en la actividad, se considera un enfoque declarativo.

Fragmento 4

Descripción

Un programa filtra productos con precios superiores a 10 dólares recorriendo una lista y comprobando cada producto uno por uno.

Clasificación

Imperativo.

Justificación

Es un ejemplo imperativo porque se especifican los pasos necesarios para obtener el resultado:

Recorrer la lista.

Obtener un producto.

Comprobar su precio.

Compararlo con 10 dólares.

Guardarlo si cumple la condición.

Continuar con el siguiente producto.

Por tanto, se está explicando **cómo realizar el proceso**.

Tabla resumen
Fragmento 1:

Paradigma: Imperativo

Motivo: Se detallan los pasos para sumar los números

Fragmento 2:

Paradigma: Declarativo

Motivo: Se indica qué datos se quieren obtener

Fragmento 3:

Paradigma: Declarativo

Motivo: Se define el resultado mediante una relación recursiva

Fragmento 4:

Paradigma: Imperativo

Motivo: Se detallan los pasos para recorrer y filtrar la lista

Identificación de paradigmas de programación a partir de ejemplos:

2.2 Actividad en grupo

Actividad seleccionada: Preparar una pizza

Para comparar los enfoques imperativo y declarativo utilizaremos una actividad cotidiana: **preparar una pizza**.

Enfoque imperativo

En el enfoque imperativo describimos detalladamente los pasos que debemos realizar.

Preparar todos los ingredientes.

Encender el horno a 220 ºC.

Colocar la masa sobre una superficie.

Extender la salsa de tomate sobre la masa.

Añadir queso.

Añadir los ingredientes elegidos.

Colocar la pizza sobre una bandeja.

Introducir la bandeja en el horno.

Hornear durante aproximadamente 10-15 minutos.

Sacar la pizza del horno.

Dejarla enfriar unos minutos.

Servir la pizza.

En este caso estamos indicando **cómo realizar la actividad paso a paso**.

Enfoque declarativo

En el enfoque declarativo nos centramos en el resultado que queremos conseguir.

Obtener una pizza cocinada, con la masa horneada y los ingredientes preparados y distribuidos sobre ella.

No se especifican todos los pasos necesarios para llegar al resultado.

El objetivo es indicar **qué queremos conseguir**, no necesariamente cómo hacerlo.

Comparación entre ambos enfoques**Imperativo vs. Declarativo**

Principal objetivo:

Imperativo: Explicar cómo realizar una tarea.

Declarativo: Explicar qué resultado queremos.

Nivel de detalle:

Imperativo: Alto.

Declarativo: Menor.

Pasos:

Imperativo: Se indican explícitamente.

Declarativo: No tienen que especificarse.

Control del proceso:

Imperativo: Mayor.

Declarativo: Menor.

Facilidad para expresar objetivos:

Imperativo: Menor.

Declarativo: Mayor.

Ejemplo:

Imperativo: Receta paso a paso.

Declarativo: Describir la pizza que queremos obtener.

Ventajas del enfoque imperativo

Permite controlar detalladamente el proceso.

Es adecuado cuando el orden de las operaciones es importante.

Facilita saber exactamente qué debe hacer el sistema.

Permite controlar el estado del programa en cada momento.

Desventajas

Puede generar código más largo.

Puede ser más complejo de mantener cuando el programa crece.

El programador tiene que preocuparse de muchos detalles del proceso.

Ventajas del enfoque declarativo

Permite expresar de forma sencilla el resultado deseado.

Puede reducir la cantidad de código necesario.

El sistema puede encargarse de decidir cómo conseguir el resultado.

Puede facilitar determinadas tareas, como realizar consultas a bases de datos.

Desventajas

Se tiene menos control sobre el procedimiento interno.

No siempre es adecuado para cualquier problema.

Puede resultar más difícil entender qué operaciones realiza internamente el sistema.

Conclusión:

Mi conclusión es que esta actividad sirve para poder plasmar las diferentes maneras de interactuar que tiene un programa dentro del ordenador, desde la parte del software hasta el hardware. Pudiendo así analizar el código fuente, el análisis léxico, sintáctico, semántico, el lenguaje tanto de bajo nivel como de medio y alto nivel que dependiendo de su patrón o paradigma de ideas se pueden usar enfoques imperativos o declarativos.

Concluyo con que el lenguaje imperativo se centra en descubrir cómo realizar una tarea mientras que el declarativo se centra en el resultado a conseguir, las dos son herramientas útiles dependiendo del objetivo de cada uno.